As a Senior Frontend Architect, I have designed the **Banking Management System** to prioritize **security, auditability, and modularity**. We will utilize a "Micro-Frontend Ready" architecture with a focus on type safety and performance.

---

1. UI Architecture

- **Stack:** Next.js (App Router) + TypeScript.
- **Rendering:** Hybrid (SSR for initial dashboards, CSR for secure interactive transaction screens).
- **Design System:** Headless UI approach using **Radix UI** for accessibility and **Tailwind CSS** for styling.
- **Security:** Content Security Policy (CSP) implementation, `httpOnly` cookies for tokens, and API request interceptors for MFA headers.

2. Folder Structure (Feature-Based)

```
src/
■■■ app/                    # Next.js App Router (Pages & Layouts)
■■■ components/             # Shared UI components
■   ■■■ ui/                 # Radix primitive wrappers (Button, Input, Modal)
■   ■■■ data-display/       # DataTables, Charts (Recharts)
■   ■■■ layout/             # Sidebar, Navbar, Breadcrumbs
■■■ features/               # Business Logic (Domain Driven)
■   ■■■ auth/               # Login, MFA, Password Reset
■   ■■■ accounts/           # Balance, Transaction History
■   ■■■ loans/              # Application forms, Review queue
■   ■■■ analytics/          # AI-driven spending charts
■■■ hooks/                  # Global hooks (useAuth, useMFA, useToast)
■■■ lib/                    # Axios instances, Auth middleware, API clients
■■■ store/                  # Zustand stores
■■■ types/                  # Shared domain types
```

3. Core Technologies

- **State Management: Zustand** (lightweight, easy to persist) + **TanStack Query** (Server state, caching, auto-refetching).
- **Forms: React Hook Form** + **Zod** (Schema validation is non-negotiable for banking).
- **API Layer:** Axios with interceptors for automatic Auth/MFA header injection.
- **Analytics/Charts: Recharts** (Customized for financial dashboards).

4. Routing & Authentication

- **Middleware:** Route protection using Next.js `middleware.ts`. Checks `JWT` token existence and session status.
- **Auth Flow:**

1. Login (Credentials).

2. Check for active session.

3. If sensitive action (Transfer > $X), trigger `MFA_REQUIRED` state.

4. Redirect to `/auth/mfa` -> Verify -> Redirect back to original route.

5. Dashboard Layout

- **Sidebar:** Persistent navigation with Role-Based Access Control (RBAC) (Customer vs. Staff vs. Admin).
- **Main Header:** Global Search (Transaction ID/Account), User Profile, Notification Bell (Fraud Alerts).
- **Dashboard Shell:**

Customer:* Summary Cards (Balances), Quick Transfer, Recent Activity.

Staff:* Loan Approval Queue (Data Grid), Pending Alerts.

6. Key Components

- **TransactionDataTable:** Built with TanStack Table, featuring server-side pagination, filtering, and row-level security masking.
- **MFAWrapper:** A Higher-Order Component (HOC) or layout wrapper that wraps any sensitive page (e.g., `/transfer`, `/loans/apply`).
- **FinancialCard:** Reusable component for visual budget indicators.

7. Form Design Strategy

Banking requires high-precision input.

- **Input Masking:** Currency formatting (`Intl.NumberFormat`).
- **Validation:** All forms defined as Zod schemas.

- **Experience:** Debounced validation for real-time feedback (e.g., checking if the destination IBAN exists before the user clicks "Continue").

8. Implementation Priorities

1. **Security First:** Implement `Axios` interceptors to handle `401` and `403` errors globally to force re-authentication or session logout.
2. **Audit Logs:** Every click on "Submit" in a transaction or loan screen must trigger a log event via a custom `useAuditLogger` hook.
3. **Data Consistency:** Utilize TanStack Query's `invalidateQueries` to ensure that after a loan application or transfer, the account balance cache is immediately updated across the UI.

Why this design works for Banking:

1. **Strict Typing:** Using TypeScript + Zod ensures that API responses (financial numbers) are never implicitly converted to strings, preventing calculation errors.
2. **Modular Features:** By separating features, the `Loan` team can iterate on their forms without breaking the `Transaction` ledger.
3. **Scalable Auth:** The middleware + interceptor pattern allows for effortless integration of future MFA providers (Biometrics, WebAuthn).